

PROJECT README & TECHNICAL SPECIFICATION

Project Name: FreshFind – Fresh All Along

Theme: eGreen Basket | **Category:** Web Innovation Unleashed

System Architecture: Frontend Single Page Application (SPA) built using React 19 + Vite, running purely on client-side logic with pre-populated JSON datasets and browser storage, adhering strictly to the zero-backend constraint of the TechWiz competition.

1. PROBLEM DEFINITION & BACKGROUND

1.1. Background & Necessity

Farmers markets play a vital role in bridging local communities with regional growers who harvest fresh, healthy, seasonal produce for local households. However, in reality, information regarding market schedules, geographic locations, operating hours, and produce availability remains fragmented across paper flyers, community physical bulletin boards, social media groups, and word of mouth. There has been no single, consolidated, and reliable digital platform for neighborhood residents to access this information seamlessly.

1.2. Key User Pain Points

- **Unpredictable Operating Hours:** Unlike conventional supermarkets, farmers markets operate on sporadic schedules (e.g., only Saturday mornings, alternate Sundays, or specific weekday afternoons). Residents struggle to plan their visits without knowing whether a market is currently open.
- **Produce Availability Uncertainty:** Shoppers rarely know in advance which fresh fruits, herbs, or vegetables are currently in season or supplied at particular market stalls.
- **Navigation & Proximity Hurdles:** Consumers often find it inconvenient to calculate travel distances, view map directions, or discover newer markets located nearest to their current whereabouts.

1.3. Proposed Solution (FreshFind)

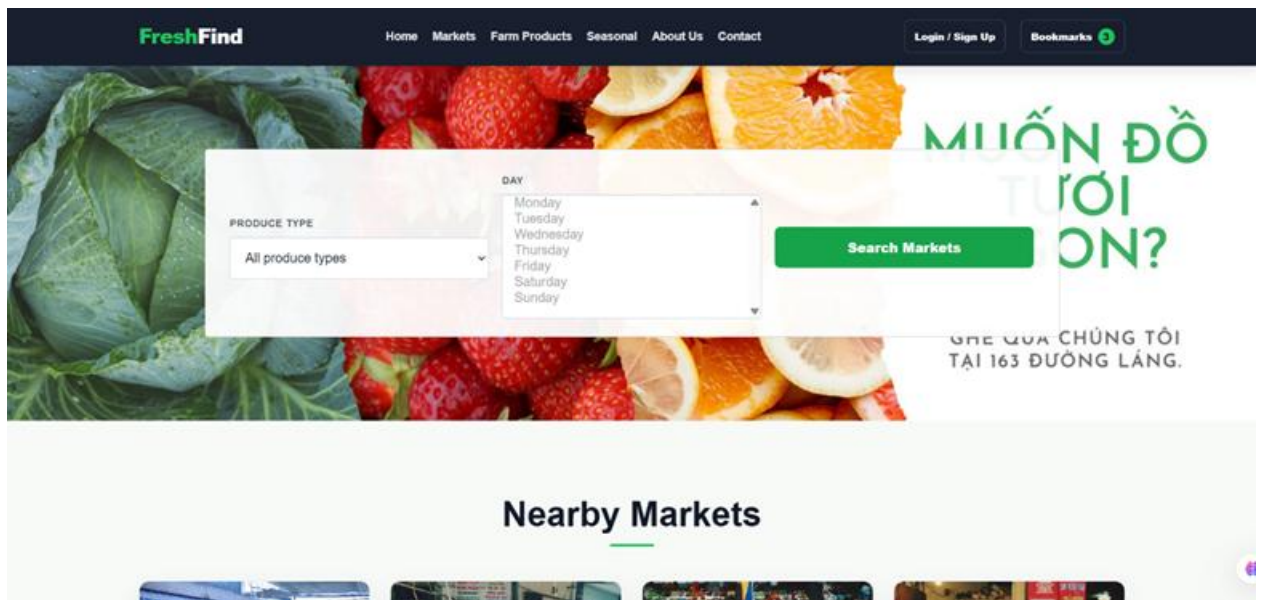
FreshFind(https://github.com/phongtoquach/freshFind_web) addresses these challenges by delivering an accessible, browser-based Single Page Application (SPA) designed to consolidate farmers market information into one unified hub:

- Aggregating real-time market details: exact street addresses, weekly operating timetables, interactive maps, and cataloged produce inventories.
 - Providing an interactive Seasonal Produce Guide that educates consumers on peak harvesting periods and links them directly to nearby markets stocking those items.
 - Leveraging Browser Geolocation to compute real-time distances (via the Haversine formula) and surface nearest markets instantly.
 - Equipping users with a client-side Bookmarking, Session Notes, and Formatted Text Export System to plan shopping itineraries effortlessly without requiring complex server-side accounts.
-

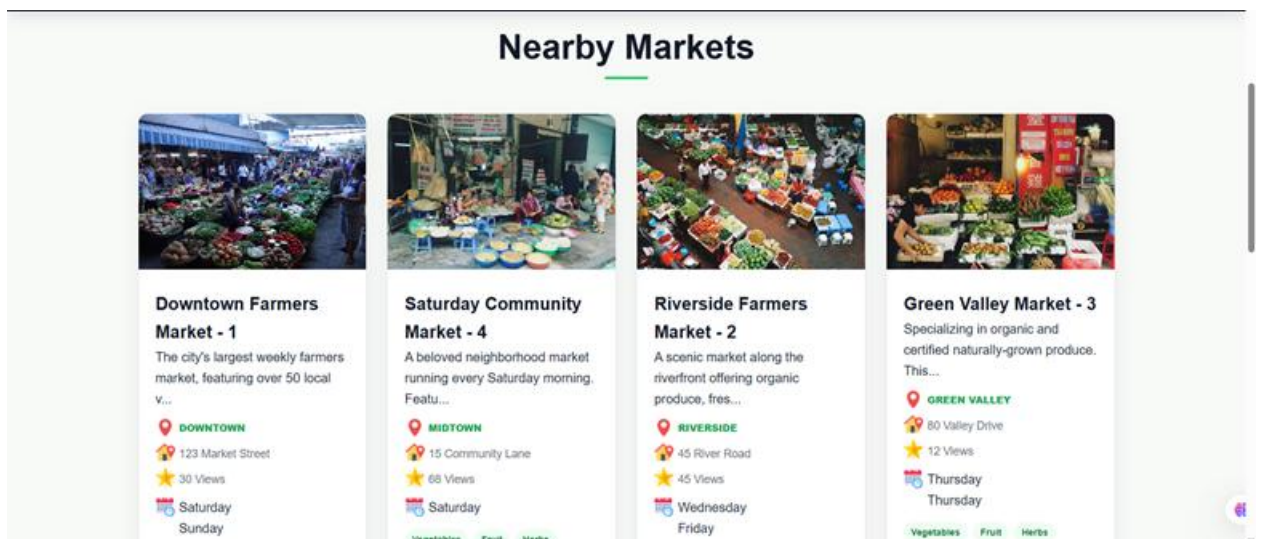
2. DESIGN SPECIFICATIONS

2.1. UI/UX Design Philosophy

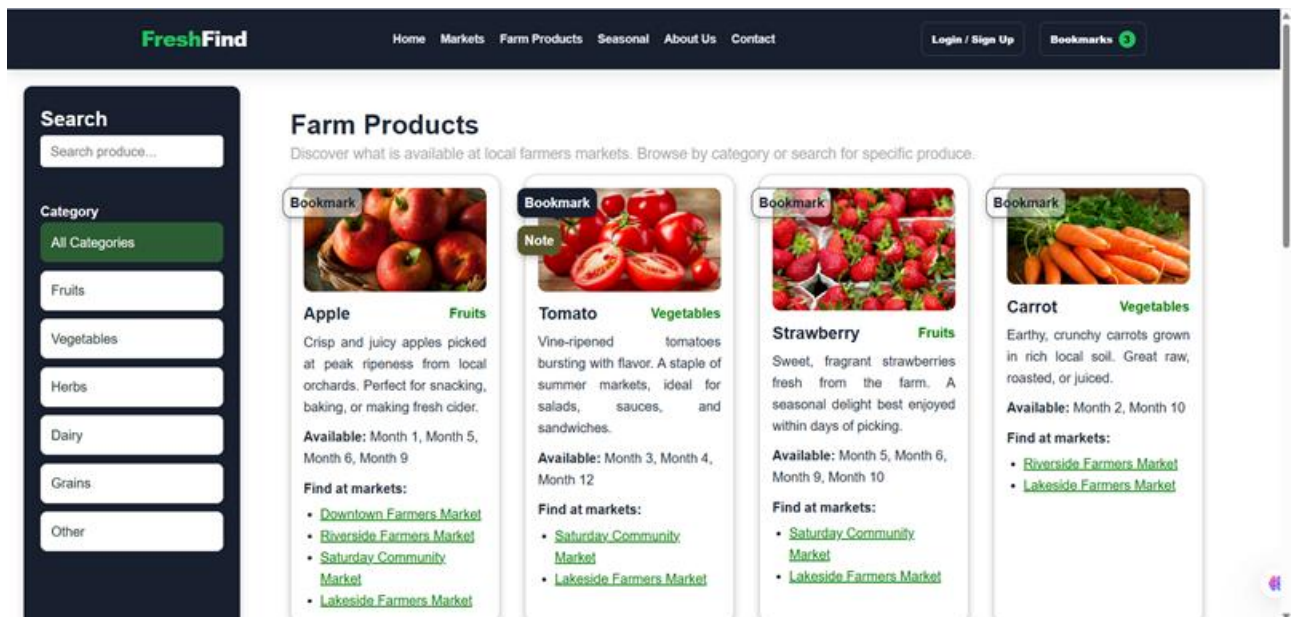
The FreshFind user interface embodies an organic, minimalist, and approachable design language. The visual identity reflects fresh agriculture through lush greens, warm harvesting ambers, and clean white backdrops that enhance readability and create an airy, positive user atmosphere. The interface is fully responsive, catering seamlessly to desktop monitors, tablets, and mobile smartphones.



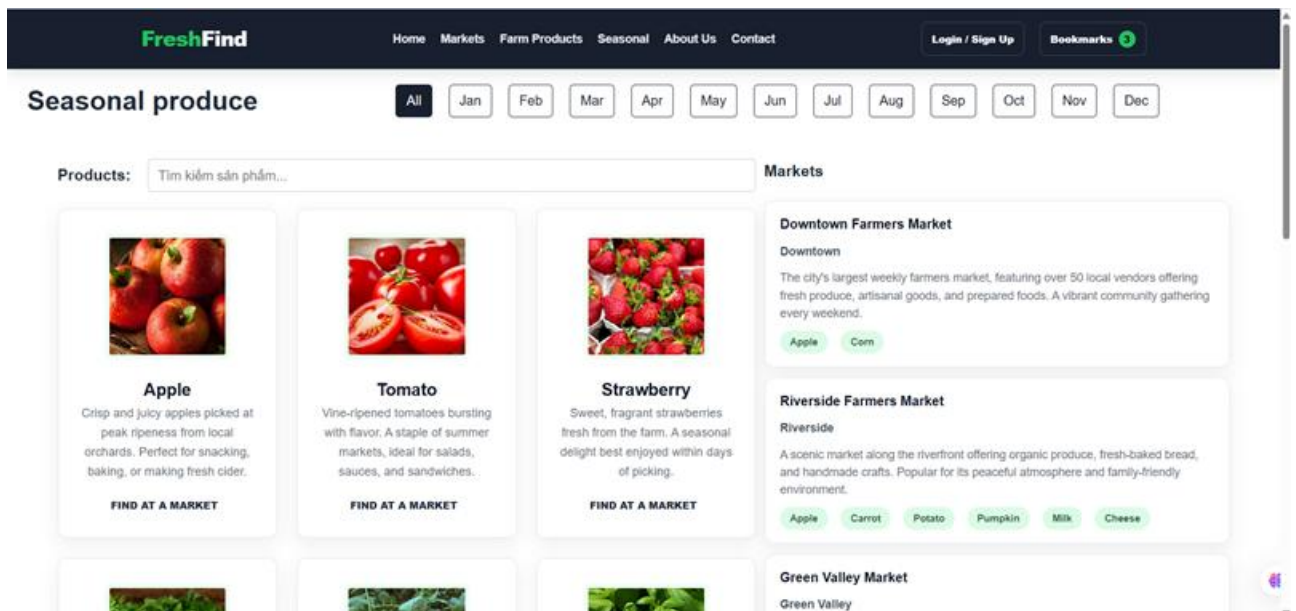
(High-fidelity Design Prototype)



(Homepage FreshFind)



(Farm products page)



(Seasonal produce page)



Saturday Community Market

Bookmark

Note

Midtown - 15 Community Lane

Open Closes at 21:00

About This Market

A beloved neighborhood market running every Saturday morning. Features local farmers, bakers, and food artisans in a relaxed community setting.

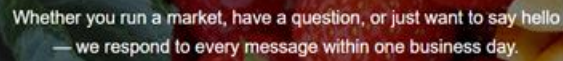
Available Produce

- Apple
- Tomato
- Strawberry
- Corn
- Milk
- Cheese

Opening Hours	
Monday	Closed
Tuesday	Closed
Wednesday	Closed
Thursday	Closed
Friday	Closed
Today! Saturday	08:00 - 21:00
Sunday	Closed

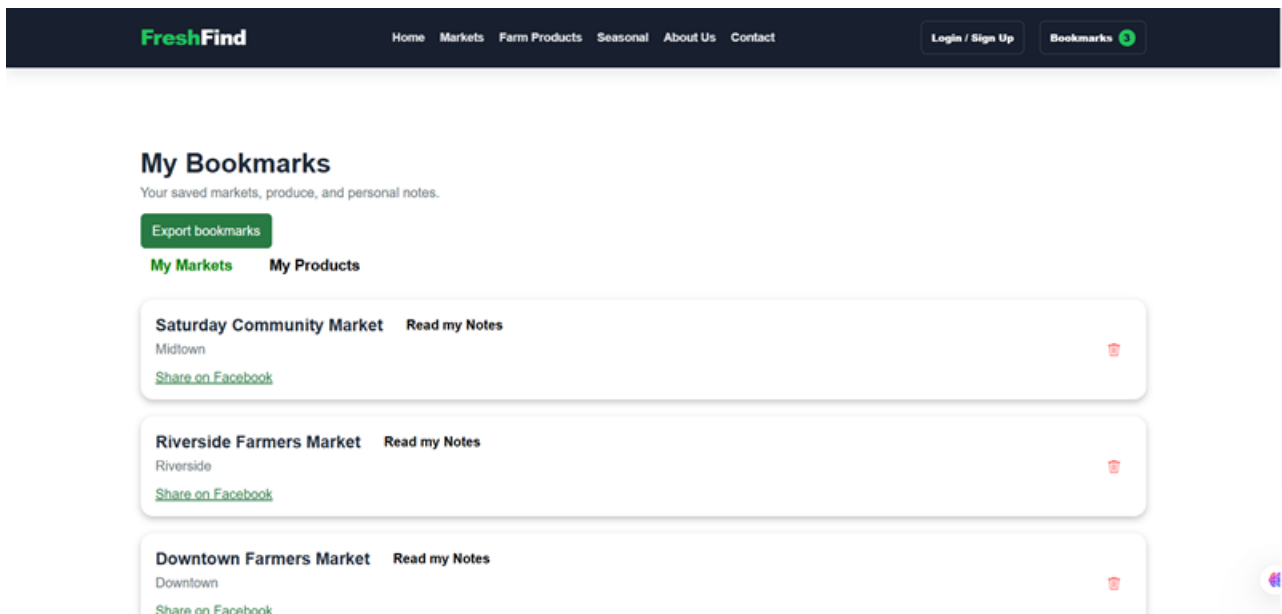


(Market details page)

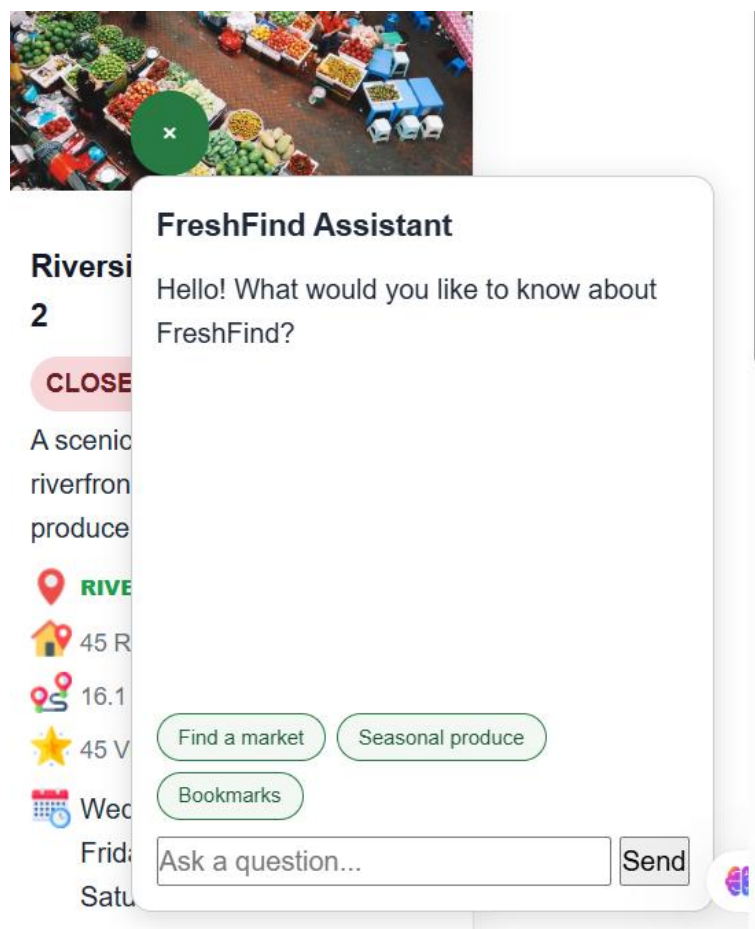


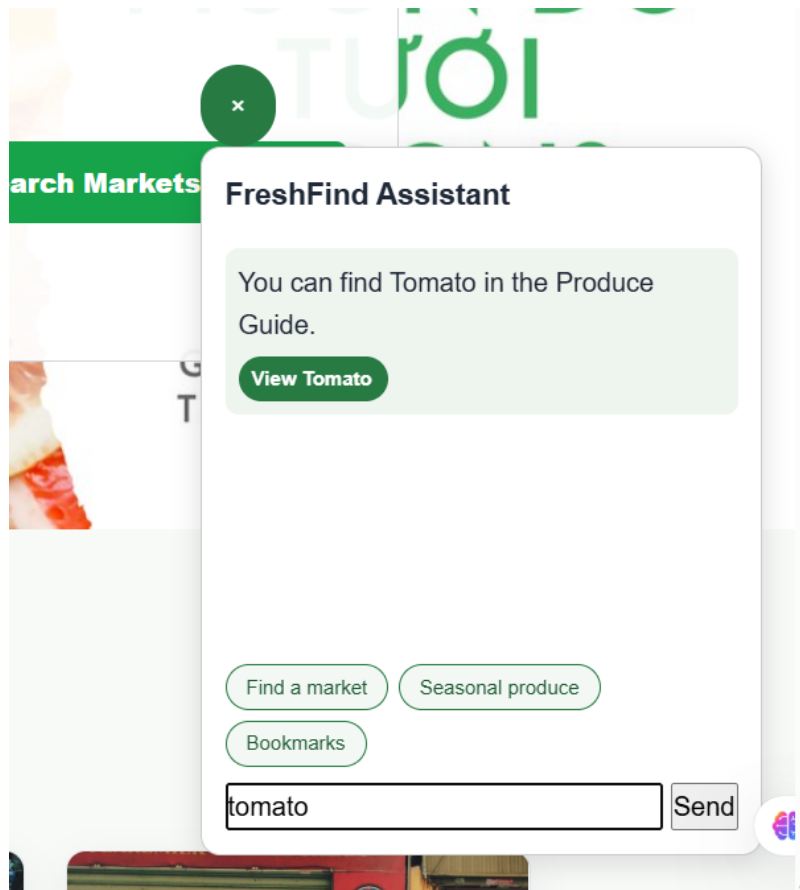
[Send Message](#)



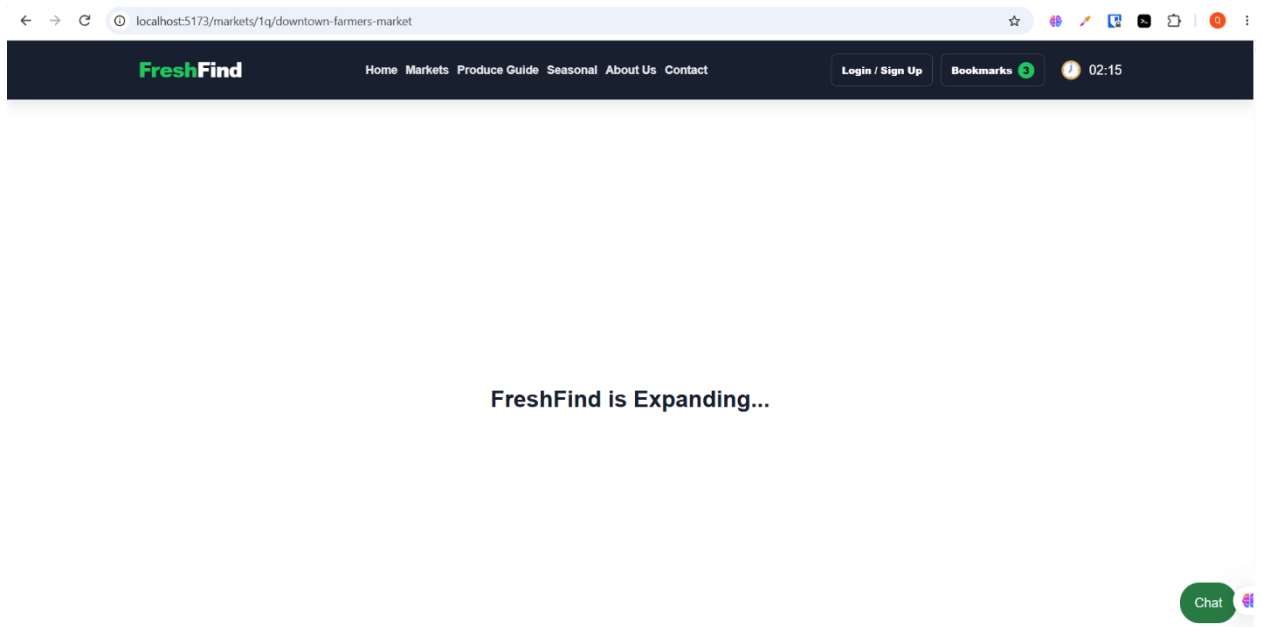


(Bookmark page)





(Chatbox search by name)



(404 showed when wrong URL)

2.2. Color Palette & Typography

Design Token	Color Code / Font Family	Design Rationale & Application
Primary Color	#2e7d32 (Forest Green) / #4caf50	Represents fresh produce, organic harvest, used for Header, primary Call-To-Action buttons, and section titles.
Secondary Accent	#fbc02d / #ffa000 (Harvest Amber)	Warm sunshine and ripe crops, utilized for "Open Right Now" live badges, seasonal highlights, and bookmark stars.
Background Canvas	#ffffff / #f8fbf8 (Clean Soft Off-White)	Ensures maximum readability, clarity, and visual contrast for food photography.
Typography Neutral	#2c3e50 / #455a64 (Dark Slate)	High-contrast text color providing strict compliance with WCAG Accessibility guidelines.
Typeface Family	'Segoe UI', system-ui, -apple-system, sans-serif	Clean, modern sans-serif typography ensuring legibility across Windows, macOS, Android, and iOS devices.

2.3. Design References & Live Prototype

Design Template Source: My team is designing based on the template at the following URL: <https://freshfind-market.youware.app/markets/downtown-farmers-market>

06:22:43 PM Saturday, Sep 26

Tìm chợ. Tươi mỗi ngày.

Discover fresh, local produce and traditional markets near you. Find the freshest vegetables, fruits, and ingredients for your kitchen.

AREA

PRODUCT

DAY

Downtown, Ark

Vegetables, Fruit

Any day

Search Markets

Use My Location

Browse by Category

Fresh produce organized for easy discovery

Vegetables

Fruits

Seafood

Meat

Herbs & Spices

What's Fresh Now

Seasonal picks from local farms -- here is what you should look for at your nearest market.

Apples

Tomatoes

Strawberries

Carrots

Basil

Popular Markets

Discover farmers markets in your area



Downtown Farmers Market

Downtown - 113 Market Street

Closed for today

Fruits

Vegetables

Meats

View Market



Riverside Farmers Market

Riverside - 41 River Road

Closed for today

Vegetables

Fruit

Meats

View Market



Green Valley Market

Green Valley - 80 Valley Drive

Closed for today

Vegetables

Fruit

Meats

View Market

View All Markets

Currently Open

Markets that are open right now -- perfect for a quick visit.

No markets are open right now. Check back during market hours!

How FreshFind Works

Finding fresh, local food has never been easier

1

Find Markets

Search by location, name, or browse our directory to find farmers markets near you.

2

Check Freshness

See what is currently in season and which markets carry the produce you want.

3

Visit & Enjoy

Get directions, check hours, and head to your nearest market for the freshest local food!

Ready to explore?

Browse all farmers markets in your area and find the freshest local produce.

Browse Markets

Explore Produce Guide

FreshFind

helping you discover fresh, local produce and the farmers markets that bring them to your neighborhood

EXPLORE

[Market Directory](#)[Produce Guide](#)[Seasonal Picks](#)[My Bookmarks](#)

COMPANY

[About Us](#)[Contact](#)

GET IN TOUCH

113 Market Street, Downtown

hello@freshfind.com

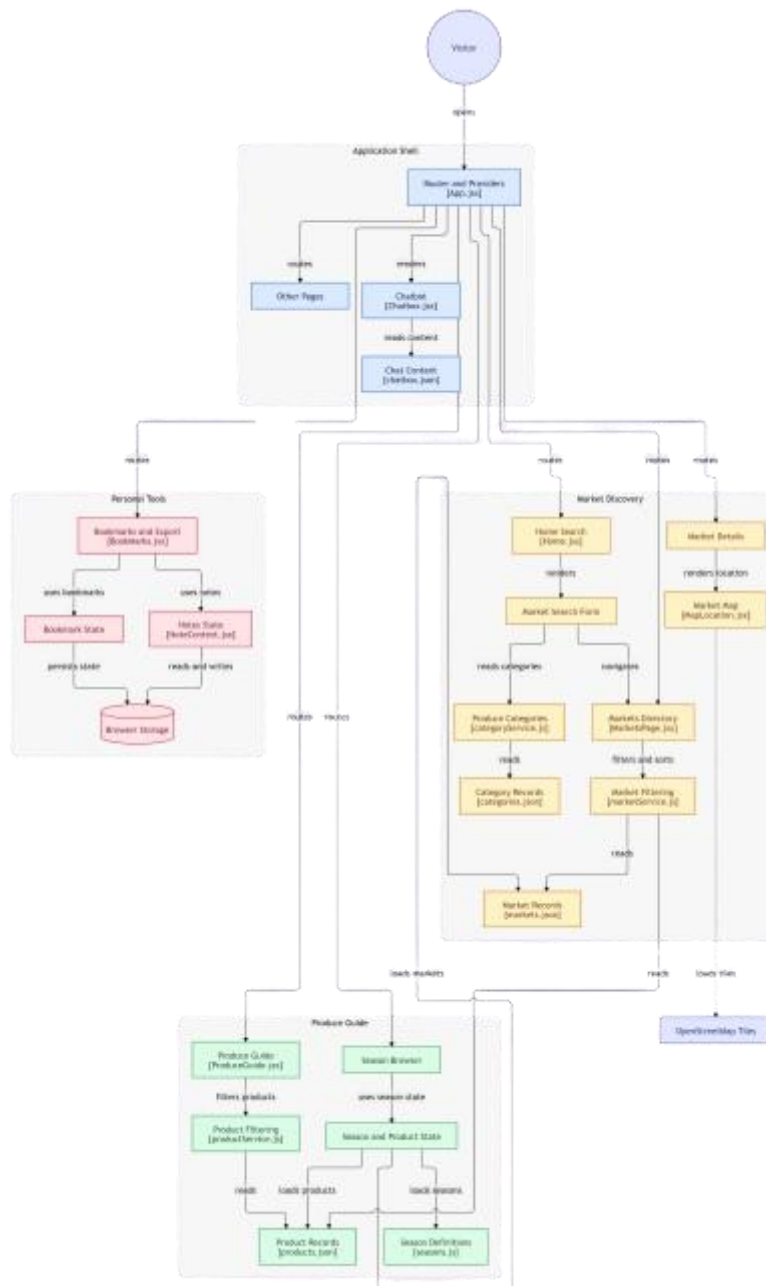
+1 (214) 567-6900

© 2016 FreshFind. Made with ♡ for the community.

A community project for the Apache FoodHub competition.

3. DETAILED SPECIFICATION OF EXISTING WEB FEATURES

3.1. Diagram repo:



3.2. Global Navigation & Utilities

- **Header Navigation (NavBar):** Hosts the brand identity "FreshFind", primary navigation links (*Home, Markets, Farm Products, Seasonal, About Us, Contact*), a live bookmark badge displaying the current number of saved items, and a dummy Login / Sign Up action button.
- **Browser Geolocation & Proximity Calculation (Haversine Formula):** Integrated within **AppContext** to request user GPS

permission via

`navigator.geolocation.getCurrentPosition()`.

Calculates the real-time geographic distance in kilometers (km) between the user's coordinates and each market's coordinates using the spherical Haversine formula ($R = 6371$ km).

- **Simulated Page View / Visitor Counter:** Located in the footer, this counter uses JavaScript coupled with `localStorage` to dynamically track and display cumulative page visit counts across user sessions.
- **Real-Time Schedule Status Checker:** An automated algorithm that compares client system time (day of week and current `HH:mm`) against each market's timetable to immediately indicate whether it is "Open Right Now" or "Closed".
- **Breadcrumb Navigation:** Renders clear hierarchical location paths (e.g., `Home / Markets / Downtown Farmers Market`) for seamless user orientation.

3.3. SEO-Friendly Slug & Dynamic Entity Lookup (`/:id/:slug`)

- **Human-Readable, SEO-Friendly URL Structure:** Markets and detailed product resources utilize an intuitive slug routing pattern: `/markets/:marketId/:marketSlug` (for example: `/markets/1/downtown-farmers-market`).
- **Flexible Entity Lookup (`getMarketByIdOrSlug`):** The service layer can resolve market entities either by numeric identifier (`id`) or by textual slug string (`slug`). This enables easy social link sharing, bookmark preservation, and enhanced search engine optimization.
- **Dual-Entity Lookups:** Enables quick cross-referencing between products and markets; clicking a produce item or market link seamlessly resolves the corresponding target entity.

3.4. Home Page (`/`)

- **Hero Section Quick Find (`HomeMarketSearchBox`):** A prominent search widget allowing visitors to select a Produce Category (dynamically populated from `categories.json`) and multiple Days of the Week. Submitting the form navigates directly to the Market Directory with serialized URL search parameters.
- **Nearby Markets Showcase (`Automatic Nearest-First Geolocation Sorting`):** A flagship feature on the FreshFind homepage designed to connect local consumers directly with neighborhood growers in their immediate vicinity. Upon initial page mount, `Home.jsx` triggers `AppContext.refreshUserCurrentLocation()` to request the user's live GPS coordinates via

`navigator.geolocation.getCurrentPosition()`. While coordinates are being queried, a smooth loading placeholder (`<div style="text-align: center;">Loading...</div>`) prevents layout shifts. Once latitude and longitude are acquired, the system computes the exact great-circle distance between the user and every market in `markets.json` using the spherical Haversine formula ($R = 6371 \text{ km}$). The markets are automatically reordered in ascending order of physical distance (`distanceA - distanceB`), placing the closest venues at the very front of the grid. Each market card renders a formatted proximity badge (e.g., `"450 m"` or `"2.4 km away"`). If GPS access is declined or unavailable, `userLocation` resolves to `null`, and the section gracefully falls back to default catalog ordering without interrupting the browsing experience.

- **Featured Currently Open Markets:** A filtered view that exclusively surfaces markets operating right at the time of browsing (`onlyOpenNow: 1`), sorted by promotional prominence (`clickCount` descending).
- **This Week's Seasonal Picks:** An automated harvest recommendation showcase powered by `ProducesGrid.jsx` (invoked with `getByCurrentMonth={1}`) that dynamically presents fresh produce items harvested in the active calendar month, complete with produce photos, descriptions, and links to local markets stocking them.

3.5. Market Directory (`/markets`)

- **Sidebar Multi-Criteria Filter:**
 - *Area Search:* Text input for neighborhood or district names (e.g., "Downtown", "Westside").
 - *Produce Category:* Dropdown selecting specific food types (Fruits, Vegetables, Herbs, Dairy, etc.).
 - *Days of Week:* Multi-select control to filter markets active on chosen weekdays.
 - *"Remove Filters" Button:* Clears all active filters and restores the full directory catalog.
- **Sorting Options:** Supports 4 distinct ordering algorithms:
 1. **Nearest First:** Evaluates GPS distance and lists nearest venues first.
 2. **Name A-Z:** Alphabetical ascending sort.
 3. **Name Z-A:** Alphabetical descending sort.
 4. **Open next day:** Prioritizes markets holding sessions on the nearest forthcoming day.

- **Market Card Grid:** Displays thumbnail images, market name, calculated distance badge (e.g. "2.4 km away"), neighborhood area, working hours summary, and direct link to full market details.

3.6. Market Details Page (`/markets/:marketId/:marketSlug`)

- **Real-Time Schedule Evaluation & Next Open Day:** Accurately determines whether the market is open during the current visit. If closed, the algorithm triggers `getNextOpenDay()` to calculate the next upcoming operating date (e.g., "Closed today • Next open: Saturday at 08:00 AM").
- **Weekly Schedule Table:** A full 7-day breakdown displaying daily start and closing hours or "Closed" badges.
- **Interactive Leaflet Map:** Embedded OpenStreetMap powered by Leaflet, displaying an exact coordinate pin (Marker) and an informational Popup containing the market title and street address.
- **Available Farm Produce Grid:** Displays farm-fresh goods available at this specific market, cross-referenced from `products.json` via `productIds`.
- **Bookmark & Personal Notes System:** Includes an instant bookmark toggle and a "Read my Notes" button that launches an interactive Modal to view, draft, and delete customized shopping notes.

3.7. Produce Guide (`/produce-guide`)

- **Instant Keyword Search:** Real-time filtering matching against produce titles and descriptions.
- **Category Filter Tabs:** Quick switches for All Categories, Vegetables, Fruits, Herbs, Dairy & Eggs, Bakery, Honey & Preserves, etc.
- **Produce Information Cards:** Presents vivid food photography, descriptive nutritional/culinary notes, ideal harvesting seasons, and **direct links to all local markets stocking this item.**
- **Item Bookmarks & Notes:** Allows saving individual produce items to personal favorites alongside private notes.

3.8. Seasonal Produce (`/products-seasons`)

- **12-Month Switcher:** Horizontal selector enabling users to inspect produce harvested in any month (January to December) or view the full annual cycle.
- **Seasonal Produce Ingestion:** Displays items matching the selected month via the `availableMonths` dataset attribute.

- **Related Markets Discovery:** Automatically queries and presents all local markets that currently stock the seasonal items on display.

3.9. Bookmarking & Notes Management (`/bookmarks`)

- **Tabbed Layout:** Segregated tabs for "My Markets" and "My Products".
- **Quick Delete Action:** Dedicated remove button on each saved item.
- **Empty State Handling:** Informative placeholder screen directing visitors back to the directory when no bookmarks have been saved yet.
- **Personal Notes Modal:** Interactive popup allowing users to add dated notes, review thoughts, and remove individual entries seamlessly.
- **Export Bookmarks via Formatted `.txt` File Download:** A dedicated "Export bookmarks" button on the `/bookmarks` page triggers an instant, purely client-side data export. The handler (`exportBookmarks()` in `BookMarks.jsx`) compiles all saved markets (numbered with names, areas, and canonical slug URLs: `${origin}/markets/:id/:slug`) and all bookmarked produce items (numbered with direct search query URLs) into a structured plain text document. It constructs an in-memory `Blob([content], { type: 'text/plain;charset=utf-8' })` and triggers an automatic browser file download named `freshfind-bookmarks.txt` via `URL.createObjectURL()`, requiring zero backend server interaction.

3.10. Contact Us & About Us (`/contact`, `/about`)

- **About Us:** Narrative describing FreshFind's community mission and support for sustainable agriculture.
- **Contact Us with Full Form Validation:** Includes contact coordinates, hours, Leaflet headquarter map, and an interactive contact form validated with real-time feedback (Full name ≥ 2 chars, strict Email regex, inquiry dropdown, message body ≥ 10 chars, touched/errors lifecycle states).

3.11. Dummy Authentication Prototype (`/login`, `/signup`)

- Clean login and sign-up user interfaces providing aesthetic and functional prototype completeness (UI Continuity) as requested by SRS Section 1.8.

3.12. Dedicated Specification: Simulated Page View & Visitor Counter (`Footer.jsx`)

To fulfill the requirement in SRS Section 1.8 ("*Visitor Counter: Simulated visitor counter using JavaScript*"), FreshFind implements an autonomous, client-side metric tracker without requiring an external analytics server:

- **Component Implementation:** Encapsulated directly within `Footer.jsx`, which remains visible across all website pages.
- **State & Storage Lifecycle:** Upon mounting, the component invokes `useEffect(() => { ... }, [])`. It inspects browser `localStorage` under the key `"visits"`.
- **Seeded Initial Baseline:** If no previous record exists (first-time visitor), it initializes with a default seed of `233` (via nullish coalescing operator: `localStorage.getItem("visits") ?? 233`).
- **Incremental Computation:** The counter computes `newCount = oldCount + 1` (resulting in an initial active count of `234`), persists `newCount` back to `localStorage`, and updates the React state via `setVisitCount(newCount)`.
- **UI Presentation:** Displays dynamically at the footer: `Total visits: {visitCount}`.

3.13. Dedicated Feature Specification: Homepage Proximity Engine, Haversine Distance Calculation & Automatic Nearest-First Sorting

FreshFind integrates an automated, client-side geospatial proximity engine to prioritize the geographically closest farmers markets directly upon opening the Homepage:

- **GPS Hardware Interrogation:** Managed inside `AppContext.jsx` via `refreshUserCurrentLocation()`, which calls `navigator.geolocation.getCurrentPosition()` with `enableHighAccuracy: true`, `timeout: 5000ms`, and `maximumAge: 60000ms`.
- **Mathematical Implementation (Haversine Formula):** Encapsulated in `calculateDistance(lat1, lon1, lat2, lon2)` within `marketService.js`, computing great-circle spherical distance using Earth radius $R = 6371 \text{ ext}\{\text{km}\}$:

```
•  
• function calculateDistance(lat1, lon1, lat2, lon2)  
  {  
•   const R = 6371; // Earth radius in kilometers
```

- `const toRad = (degree) => degree * Math.PI / 180;`
- `const dLat = toRad(lat2 - lat1);`
- `const dLon = toRad(lon2 - lon1);`
- `const a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +`
- `Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.sin(dLon / 2) * Math.sin(dLon / 2);`
- `const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));`
- `return R * c;`
- `}`

- **Proximity Badge Formatting (`formatDistance`):** In `marketService.js`, distances are formatted intelligently for readability:

- `export function formatDistance(distanceKm) {`
- `if (distanceKm < 1) {`
- `return `${Math.round(distanceKm * 1000)}`
- `m`;`
- `}`
- `return `${new Intl.NumberFormat("en-US", {`
- `maximumFractionDigits: 1`
- `}).format(distanceKm)} km away`;`
- `}`

In `MarketsGrid.jsx`, each card renders this badge alongside a pin icon (``).

- **Automated Nearest-First Sorting on Homepage:** In `Home.jsx`, the "Nearby Markets" section passes `sortBy="nearest_first"` and `userCurrentLocation` to `MarketsGrid`. The sorting routine computes distances on-the-fly and organizes the array by ascending numerical value (`distanceA`

- `distanceB`), placing the closest market venue at the top-left slot of the homepage grid.
- **Graceful Degradation Fallback:** If the user declines geolocation permissions, if GPS is unavailable, or if the request times out, `userLocation` defaults to `null`. The component immediately terminates the loading state, hides the distance badges, and preserves the default catalog display order without throwing exceptions or breaking the UI.

3.14. Dedicated Feature Specification: Real-Time Header Clock & Dynamic Timetable Synchronization (`Header.jsx` & `dateTimeUtils.js`)

To eliminate shopper uncertainty regarding sporadic farmers market operating hours without relying on external time servers, FreshFind implements an autonomous real-time clock and synchronization engine directly on the client side:

- **Component Implementation:** Embedded in `Header.jsx` within the primary navigation actions (`.nav-actions`), positioned alongside a dedicated clock indicator icon (``).
- **Time Formatting Algorithm (`getFormattedCurrentHourMinute`):** Encapsulated in `src/utils/dateTimeUtils.js`. The routine creates a new `Date()` instance, queries current hours (`nowObj.getHours()`) and minutes (`nowObj.getMinutes()`), and applies `String.prototype.padStart(2, '0')` to guarantee strict two-digit 24-hour military notation (`HH:mm`, e.g., `09:05`, `14:30`).
- **Cross-Subsystem Synchronization Engine:** The client device's clock serves as the single source of temporal truth driving real-time business logic across the web application:
 - **Market Card Live Status Badges (`isMarketOpenNow` in `marketService.js`):** Queries the active weekday via `nowObj.getDay()` to retrieve the corresponding entry in `market.schedule`. Converts opening and closing time strings into absolute minutes since midnight using `timeToMinutes(HH:mm) = hours * 60 + minutes`. Evaluates `currentMinutes >= startMinutes && currentMinutes <= endMinutes`, dynamically displaying a green "Open Right Now" badge or a red "Closed" badge on each market card in `MarketsGrid.jsx`.

- **Homepage "Featured Currently Open Markets":** Dynamically applies the filter flag `onlyOpenNow: 1` in `getMarketsByFilters()`, filtering out closed venues and isolating markets actively welcoming visitors at the exact browsing moment.
- **Market Details Timetable Evaluation** (`MarketDetailsPage.jsx`): Computes `isWithinOpenHours` by verifying `!isTimeBefore(currentTime, todaySchedule.hours.start) && isTimeBefore(currentTime, todaySchedule.hours.end)`. Displays "Open • Closes at [HH:mm]" or invokes `getNextOpenDay()` to cycle through the upcoming 7 calendar days to predict "Closed today • Next open: [Day] at [Start Hour]".
- **Seasonal Harvest Produce Alignment:** Evaluates the active calendar month (`new Date().getMonth() + 1`) to power the "This week's seasonal picks" harvest showcase on the homepage.
- **Zero-Backend Compliance:** Executes 100% within the browser runtime environment, incurring zero network latency, eliminating external NTP API dependencies, and respecting the user's localized timezone natively.

3.15. Dedicated Feature Specification: Client-Side Rule-Based AI Assistant Chatbot (`Chatbox.jsx` & `chatbox.json`)

To assist consumers in navigating the platform, discovering market schedules, and checking produce availability without external cloud AI subscriptions, FreshFind integrates an in-browser deterministic rule-based AI Assistant chatbot:

- **Component Implementation:** Encapsulated in `Chatbox.jsx` and rendered globally in `App.jsx` above the footer, ensuring constant availability across all routes. Styled via `chatbox.css` as a floating widget fixed at the viewport's bottom-right (`position: fixed; right: 20px; bottom: 20px; z-index: 1000`).
- **Unified Dynamic Knowledge Graph** (`searchableChatbotData`): At module startup, the chatbot synthesizes static curated intents (`chatbox.json`) with dynamic entity catalogs extracted from `products.json` (every fruit/vegetable) and `markets.json` (every market venue). This

creates an extensive, self-updating knowledge base requiring zero manual synchronizations.

- **Longest-Match Heuristic Scoring Algorithm:** Sanitizes user queries via lowercase normalization, scans for keyword occurrences, and assigns a match weight proportional to keyword character length (`score = keyword.length`). Specific multi-token entity matches (e.g., "Downtown Farmers Market") automatically outrank broad generic matches (e.g., "market").
- **Deep Navigation with Modal Auto-Dismiss:** Responses provide actionable direct navigation links (using React Router's `<Link>`). Clicking any recommendation automatically invokes `closeChat()`, smoothly routing the visitor to the destination market or produce catalog while cleaning up chat state.

4. APPLICATION WORKFLOW & ARCHITECTURE

4.1. System & Architectural Data Flow

FreshFind adopts React's Unidirectional Data Flow combined with React Context Providers and Browser LocalStorage:

1. **Application Bootstrapping:** At startup, `App.jsx` wraps all routes within four foundational Context Providers: `AppProvider` (Geolocation GPS), `ProductsSeasonProvider` (Season & month filter state), `BookmarkProvider` (Favorite markets and produce), and `NoteProvider` (Personal notes).
2. **Data Ingestion:** Static datasets (`markets.json`, `products.json`, `categories.json`, `seasons.js`) are imported directly into memory.
3. **Client-Side Business Logic:** Geospatial distance calculation (Haversine formula), real-time schedule checks, keyword matching, and multi-criteria sorting execute on the client with zero latency.
4. **State Persistence:** User bookmarks and personal notes synchronize bidirectionally with `localStorage`, preserving user data across page refreshes and route transitions.

4.2. User Journey & Operational Workflows

- **Market Discovery Journey:** Visitor enters Home → Initiates Quick Find or browses Market Directory → Adjusts area, day, or produce filters → Selects Nearest First sort → Clicks market card to open

- SEO-friendly URL (`/markets/:id/:slug`) → Inspects timetable and Leaflet map → Toggles Bookmark or adds personal notes.
- **Seasonal Produce Journey:** Visitor enters Seasonal Produce → Selects current calendar month → Discovers in-season crops → Views list of stocking markets → Clicks market link to schedule a visit.
 - **Shopping Planning & TXT Export Journey:** Visitor navigates to Bookmarks → Reviews saved venues and goods → Launches Notes Modal to review shopping lists → Clicks "Export bookmarks" button → Browser instantly generates and downloads `freshfind-bookmarks.txt` containing formatted shopping lists with direct URLs and areas.
 - **Page View Counter Lifecycle Workflow:** Visitor accesses any page on FreshFind → `Footer` mounts in DOM → `useEffect` checks `localStorage.getItem("visits")` → Increments tally by 1 → Writes back to `localStorage` → Re-renders footer display with updated count.

4.3. Dedicated Operational Workflow: Homepage Distance Calculation & Nearest-First Market Auto-Sorting

To ensure neighborhood residents immediately discover the markets closest to them without manual searching, FreshFind implements an automated proximity ordering pipeline upon accessing the homepage:

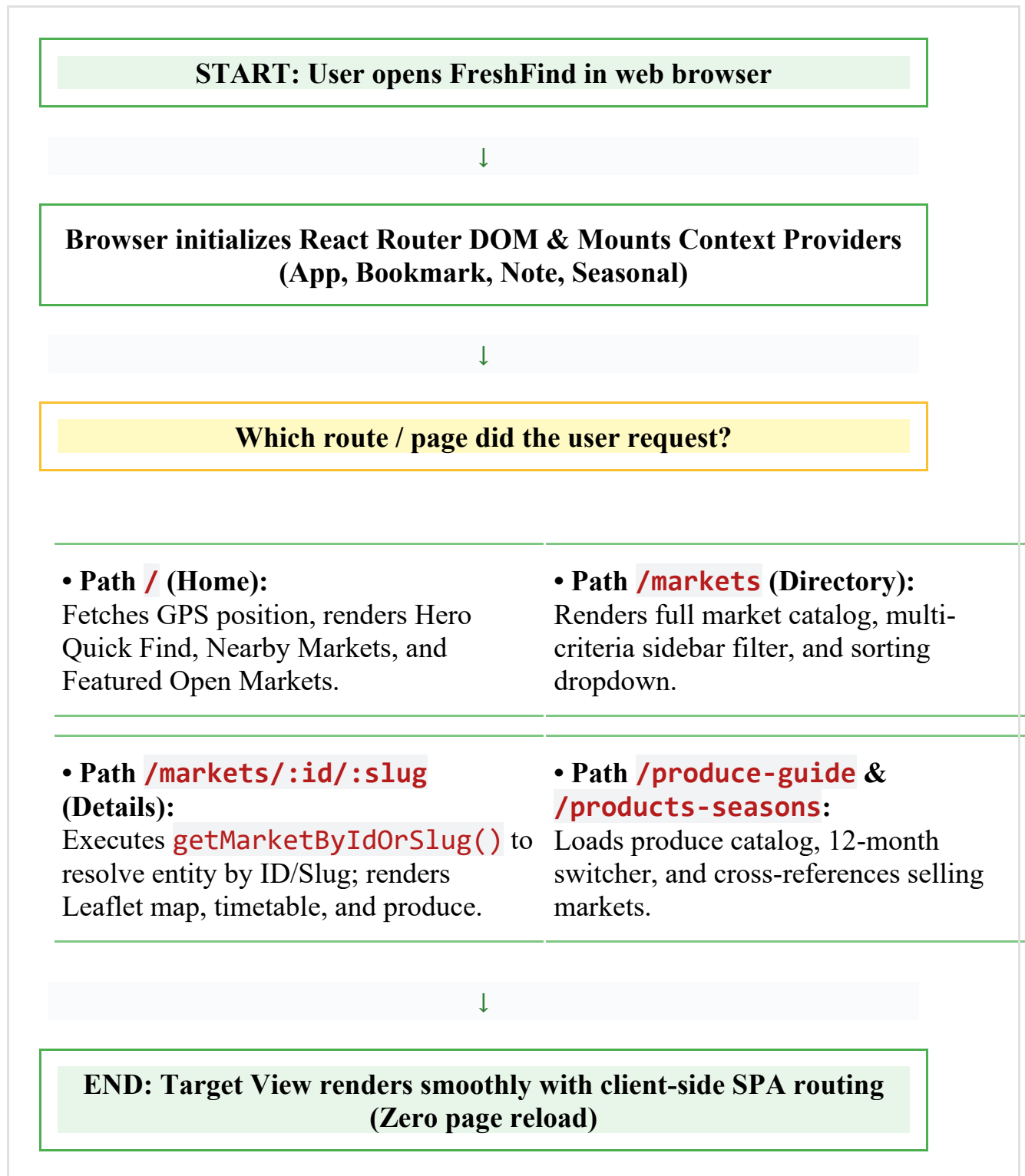
1. **Component Initialization & Mount:** When the user visits `/`, `Home.jsx` mounts. While GPS coordinates have not yet resolved, `userLocation` equals `undefined`. The component conditionally renders a clean loading indicator (`<div style="text-align: center;">Loading...</div>`) to avoid layout shifts while hardware coordinates are determined.
2. **Geolocation API Handshake:** The `useEffect` hook in `Home.jsx` executes on mount, calling `refreshUserCurrentLocation()` from `AppContext`. The browser prompts the user for location access and invokes `navigator.geolocation.getCurrentPosition()` with high-accuracy mode.
3. **Coordinate State Synchronization:** Upon receiving position coordinates, `AppContext` updates state with `{ latitude: position.coords.latitude, longitude:`

- `position.coords.longitude` }, causing `Home.jsx` to re-render with active coordinates.
4. **MarketsGrid Ingestion with `nearest_first` Parameter:** `Home.jsx` renders the "Nearby Markets" section by invoking `<MarketsGrid sortType="nearest_first" userCurrentLocation={userCurrentLocationObj} />`.
 5. **Great-Circle Distance Calculation (Haversine Formula):** Inside `marketService.js`, the engine passes the user's coordinates and iterates over every market entry in `markets.json`. It executes `calculateDistance(lat1, lon1, lat2, lon2)` with Earth radius $R = 6371 \text{ km}$, computing precise spherical distances with zero latency.
 6. **Ascending Proximity Sorting (Nearest First):** The sorting engine runs `sortMarketsByType()` with `case "nearest_first"`. It evaluates `distanceA - distanceB`, reorganizing the market array in strict ascending order of physical distance, positioning the closest farmers market at index 0.
 7. **Proximity Badge Formatting & Card Rendering:** For each sorted market, `MarketsGrid.jsx` passes the distance to `formatDistance(distanceKm)`:
 - If distance $< 1 \text{ km}$: Formatted in meters (e.g., `"450 m"`).
 - If distance $\geq 1 \text{ km}$: Formatted in kilometers with 1 decimal place (e.g., `"1.8 km away"`).
- The resulting proximity badge is rendered alongside a location pin icon on each market card.
8. **Graceful Degradation / Permission Denied Fallback:** If GPS access is declined, disabled, or times out, `userLocation` resolves to `null`. The component immediately terminates the loading state, omits distance badges, and renders the default market catalog order seamlessly without errors or application crashes.

5. COMPREHENSIVE SYSTEM FLOWCHARTS

The following 9 structured flowcharts illustrate the core business logic, algorithms, and lifecycle workflows powering the FreshFind web portal:

5.1. Flowchart 1: Site Navigation & Dynamic Route Flowchart (`/:id/:slug`)



5.2. Flowchart 2: Market Search, Multi-Criteria Filter & Haversine Distance Sort



5.3. Flowchart 3: Real-Time Schedule & Next Open Day Calculation Algorithm

START: User navigates to Market Details
(`/markets/:marketId/:marketSlug`)



Retrieve current system time: Active day of week (key) and current time in HH:mm format



Is the market scheduled to open today? (`todaySchedule.open === true`)

[YES: SCHEDULED TODAY] ↓

Evaluate Time Window:

Check if (**Current Time** >= **Start Hour**) AND (**Current Time** < **End Hour**).

- *If True*: Display prominent green badge "Open Right Now".
- *If False*: Mark as "Closed for today" (before or after hours).

[NO: CLOSED TODAY] ↓

Execute `getNextOpenDay()`:

Loop through upcoming 7 calendar days to find the next day marked with **open: true**.

→ Display: "Closed today • Next open: [Day] at [Start Hour]".

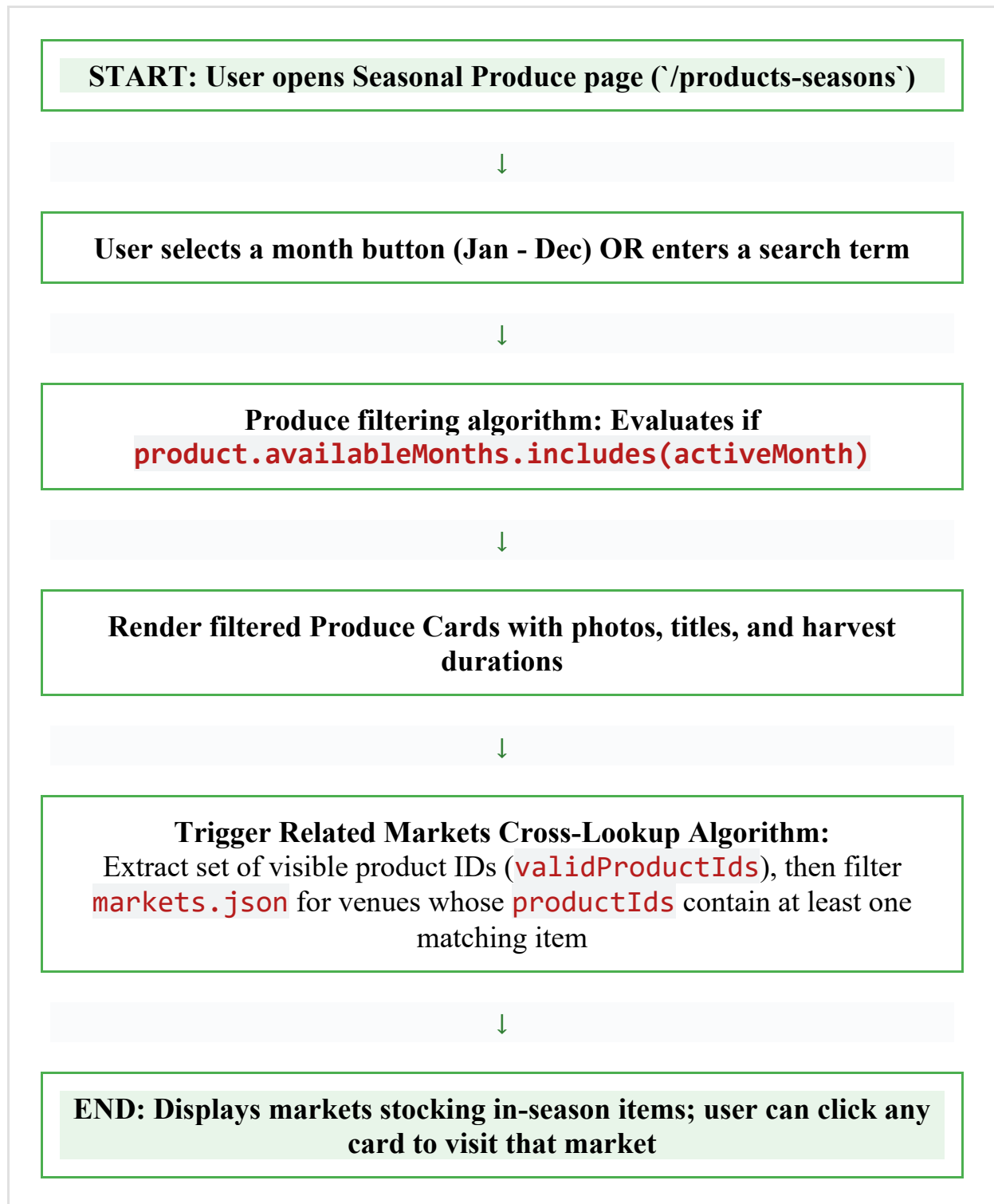


Render 7-day Weekly Schedule Table and Interactive Leaflet Map with Coordinates Marker



END: User receives accurate opening times and geographic directions

5.4. Flowchart 4: Seasonal Produce Month Selector & Related Markets Lookup



5.5. Flowchart 5: Bookmarking, Personal Notes & Formatted `.txt` File Export

START: User views a Market or Produce card across the site



Which interaction does the user trigger?

[CLICK BOOKMARK BUTTON] ↓

- If already saved: Remove ID from favorites array.
- If not saved: Add ID to favorites array.
- Updates live badge count in Navbar.
- Automatically synchronizes with `localStorage`.

[CLICK "READ MY NOTES"] ↓

- Opens the interactive Notes Modal for this specific item.
- Fetches existing notes from `NoteContext`.
- User inputs new note and submits form.
- User clicks "x" icon to delete individual notes.
- Notes persist in client storage.



User navigates to `/bookmarks` to review all saved markets and produce



Export Option: User clicks "Export bookmarks" → `exportBookmarks()` formats text → Client builds in-memory `Blob` → Browser triggers automatic download of `freshfind-bookmarks.txt`



END: User shopping itinerary and bookmarks saved locally & exported as TXT

5.6. Flowchart 6: Homepage Real-Time Geolocation, Haversine Distance Calculation & Nearest-First Market Auto-Sorting

START: User accesses FreshFind Homepage (^)



Home.jsx Lifecycle Mount: Evaluates `userLocation === undefined` → Displays temporary loading placeholder (`Loading...`)
Executes `useEffect()` → Invokes `refreshUserCurrentLocation()` via `AppContext`



Browser queries Device Location:
`navigator.geolocation.getCurrentPosition()` with `enableHighAccuracy: true`



Did the user grant GPS permission & coordinates were successfully obtained?

[YES: GPS COORDINATES AVAILABLE] ↓

1. **Synchronize Context:** Store `{ latitude, longitude }` into `AppContext` state.
2. **Render Nearby Markets Grid:** `Home.jsx` re-renders with `userLocation` defined and passes `sortType="nearest_first"` to `<MarketsGrid />`.
3. **Calculate Spherical Distances:** Iterate through `markets.json`, calling

[NO: DENIED / UNAVAILABLE / TIMEOUT] ↓

1. **Safe Fallback State:** Set `userLocation = null` gracefully.
2. **Dismiss Loading Screen:** `Home.jsx` exits loading state and proceeds to render page sections.
3. **Omit Proximity Badges:** Distance badges are cleanly omitted from market cards.
4. **Preserve Default Catalog:** Markets display in standard default order without crashing or freezing.

`calculateDistance(userLat, userLon, mktLat, mktLon)` via Haversine formula ($R = 6371 \text{ ext}\{ \text{km}\}$).

4. **Auto-Sort Nearest First:** Array reorders by ascending distance (`distanceA - distanceB`), moving closest markets to index 0.

5. **Format Proximity Badges:** Call `formatDistance(distanceKm)` to generate human-readable distance (e.g. "450 m" or "2.4 km away").

6. **Prioritized Homepage Display:** Geographically nearest markets are placed at the very front of the "Nearby Markets" section.

5. **Full Usability Intact:** Visitor can still search by area, filter by day/produce category, and access market details.



Render Subsequent Homepage Sections:

- "Featured Currently Open Markets": filtered by `onlyOpenNow: 1` & sorted by `featured_desc`
- "This week's seasonal picks": loaded via `<ProduceGrid getByCurrentMonth={1} />`



END: FreshFind Homepage renders interactively with geographically nearest markets prioritized at the front

5.7. Flowchart 7: Simulated Page View & Visitor Counter Lifecycle (`Footer.jsx`)

START: Visitor accesses or refreshes any page on FreshFind



Footer.jsx component is mounted into the DOM



Lifecycle Hook **useEffect(() => { ... }, [])** executes once on mount



Does key **"visits"** exist in browser **localStorage**?

[YES: RECORD FOUND] ↓

[NO: FIRST-TIME VISIT] ↓

Retrieve existing numerical count:

oldCount =
Number(localStorage.getItem("visits"))

Apply seeded baseline value
via nullish coalescing:

oldCount = 233



Compute increment: **newCount = oldCount + 1** (e.g. 234)



Persist updated counter: **localStorage.setItem("visits", newCount)** & Update State: **setVisitCount(newCount)**



END: Footer renders live counter **"Total visits: [newCount]"**
accurately

5.8. Flowchart 8: Rule-Based AI Chatbot Query Processing & Longest-Match Entity Routing (**Chatbox.jsx**)

START: User interacts with Chatbox (Clicks Suggestion Chip OR Enters Custom Text & Submits)



Input Sanitization & Normalization: `questionToSearch = normalizeText(userText.trim())`



Iterate through Unified Index `searchableChatbotData` (Curated FAQ + Dynamic Products + Dynamic Markets)



Does `questionToSearch` include any candidate's `normalizedKeyword`?

[YES: KEYWORD MATCH FOUND]

- Evaluate character length:
`longestKeyword = keyword.length`
- Assign candidate score: `item.score = longestKeyword`
- Push candidate into `matches` array.

[NO: NO KEYWORDS MATCH]

- Ignore item; continue iteration through knowledge graph.



Are any matching candidates found? (`matches.length > 0`)

[YES: RANK & RENDER ANSWERS]

[NO: FALLBACK FEEDBACK]

- Set `answers = []` and `hasAsked =`

1. Sort matches descending: `b.score` - `true`.
`a.score` (Longest keyword takes top priority).
 2. Deduplicate destination URLs
(`!linkAlreadyAdded`).
 3. Take top 4 recommendations
(`topAnswers.length === 4`).
 4. Render bot response message cards
with deep-link pill buttons.
-



User clicks recommendation link (`<Link to={item.link}>`) →
Triggers `closeChat()` & SPA routes directly to destination



END: Chatbox dismisses smoothly; user arrives at target Market
Details or Produce Guide

5.9. Flowchart 9: Real-Time Clock Evaluation & Cross-Component Schedule Synchronization (`Header.jsx` & `dateTimeUtils.js`)

START: Application Mounts in Web Browser (`App.jsx` & `Header.jsx`)



`Header.jsx` invokes `getFormattedCurrentHourMinute()` from `dateTimeUtils.js`



Acquire client device timestamp: `const nowObj = new Date()` →
Extract `curHour` & `curMin`



Format with Zero-Padding: `padStart(2, '0')` → Render `HH:mm` in Header alongside Clock Icon



Temporal Synchronization Across Application Subsystems

1. Market Status Badges
`isMarketOpenNow()`:
Converts `start` & `end` to minutes via `timeToMinutes()`.
Checks current minute.
→ Renders green "Open Right Now" or red "Closed".

2. Featured Open Filter
Homepage query:
`onlyOpenNow: 1` filters markets trading actively at the present minute.

3. Next Open Prediction
`MarketDetailsPage`:
If closed today, `getNextOpenDay()` loops 7 upcoming days.
→ Displays "Closed today • Next open: [Day] at [Hour]".

4. Seasonal Harvest ProducesGrid:
Matches `nowObj.getMonth() + 1` against `availableMonths`.
→ Surfacing active seasonal crops.



END: Unified, deterministic temporal coherence maintained across all client views

6. ASSUMPTIONS MADE IN THE PROJECT

In accordance with the competition guidelines and the SRS document (Sections 1.5 Constraints and 1.8 Interface Requirements), the project is engineered upon the following foundational assumptions:

- **Pure Client-Side Architecture (Zero-Backend):** Per SRS constraint 1.5, the application operates entirely without a server-side database backend. All market profiles, schedules, product catalogs, and category lists are served from local static JSON/JS files (`markets.json`, `products.json`, `categories.json`, `seasons.js`). The application treats these files as read-only.
 - **Client-Side Storage for Bookmarks, Notes & TXT Export:** User preferences, favorites, and notes are managed in client memory and persisted through browser storage (`localStorage` and `sessionStorage`), ensuring a seamless experience across page reloads without requiring server user accounts. The bookmark export feature is handled 100% in-browser via the W3C `Blob` and `URL.createObjectURL()` APIs.
 - **Device Real-Time Clock Reliability:** Operating status indicators ("Open Right Now" vs. "Closed") rely on the user device's local system time to match the market's operating hours.
 - **Simulated Visitor Metrics:** In the absence of a global analytics server, the visitor counter in the footer is simulated via client-side JavaScript and updated per session in `localStorage` as suggested by SRS Section 1.8.
 - **OpenStreetMap & Leaflet Integration:** Interactive location mapping utilizes Leaflet.js with OpenStreetMap tiles, ensuring accurate spatial visualization and routing coordinates without external commercial API key restrictions.
 - **Dummy Authentication Prototype:** Login and registration pages are functional prototypes designed to complete the user journey (UI Continuity) as requested by SRS Section 1.8.
-

7. SYSTEM REQUIREMENTS & INSTALLATION INSTRUCTIONS (MANDATORY)

Follow the step-by-step instructions below to set up and run the FreshFind application on a local development environment:

7.1. Prerequisites:

- **Node.js Runtime:** Version 18.x, 20.x, or newer (bundled with `npm` package manager).
- **Web Browser:** Google Chrome, Microsoft Edge, Mozilla Firefox, or Apple Safari (latest stable release).
- **Display Resolution:** Fully responsive; optimized for mobile (375px+), tablet (768px+), and desktop (1024px+).

7.2. Step-by-Step Installation:

1. **Step 1: Extract Project Source Code:** Extract the submitted `SourceCode.zip` archive or clone repository from https://github.com/phongtoquach/freshFind_web.git into your desired local directory.
2. **Step 2: Open Terminal / Command Prompt:** Open your Terminal, Command Prompt, or PowerShell, and navigate to the project root directory:

```
cd path/to/freshfind-web
```

3. **Step 3: Install Project Dependencies:** Execute the following command to download and install all necessary packages specified in `package.json`:

```
npm install
```

4. **Step 4: Start the Local Development Server:** Launch the Vite development server by running:

```
npm run dev
```

5. **Step 5: Access the Application:** Open your web browser and navigate to the local URL displayed in the terminal output:

```
http://localhost:5173/
```

6. **Step 6 (Optional): Build Production Bundle:** To compile and preview an optimized production build, run:

```
7.  
8. npm run build  
9. npm run preview
```

8. AI CHATBOT RULE - BASE & REAL-TIME INTERACTION ENGINE

FreshFind integrates an autonomous, client-side **Rule-Based AI Assistant Chatbot** (`Chatbox.jsx`) coupled with a continuous **Real-Time System Clock** (`Header.jsx` & `dateTimeUtils.js`). Operating strictly under the TechWiz zero-backend constraint, this subsystem delivers instantaneous guidance, deep entity discovery, and temporal schedule verification without incurring cloud AI API expenses or latency overhead.

8.1. Architectural Philosophy & Zero-Backend Compliance

Modern web users expect interactive conversational support, yet integrating commercial Large Language Model (LLM) APIs introduces latency bottlenecks, privacy leakage, recurring API subscription costs, and breaking dependencies when network connectivity is degraded or server quotas expire. In direct alignment with the TechWiz competition mandate (pure frontend Single Page Application with zero backend), FreshFind adopts a deterministic, rule-based inference architecture:

- **Sub-Millisecond Response Latency (< 1ms):** Because all Q&A intent sets and catalog entities reside in local browser memory, matching occurs synchronously in near-zero time without round-trip network delays.
- **100% Deterministic & Hallucination-Free:** Every response is derived exclusively from verified catalog entries and pre-approved navigational routes, eliminating inaccurate or misleading recommendations.
- **Offline & Resource-Light:** Requires zero external API tokens (such as OpenAI or Gemini API keys), ensuring frictionless offline operation and zero infrastructure hosting expenses.

8.2. Dual-Layer Knowledge Base Architecture (`searchableChatbotData`)

Rather than relying on a static, inflexible FAQ document, the FreshFind Assistant combines structured editorial intent rules with dynamically synthesized catalog records:

Knowledge Tier	Source Dataset & Entities	Dynamic Mapping & Response Rationale
Tier 1: Curated Intent FAQs	<code>src/data/chatbox.json</code> 6 foundational intent clusters: • Market Directory discovery • Opening hours lookup • Produce guide exploration • Seasonal calendar query • Bookmarks & notes assistance • Customer support & contact	Provides high-level navigational directions and educational answers. Each entry specifies trigger keywords, a clear instructional answer, and a canonical route destination (e.g., <code>/markets</code>, <code>/produce-guide</code>, <code>/products-seasons</code>, <code>/bookmarks</code>, <code>/contact</code>).
Tier 2A: Farm Produce Index	<code>src/data/products.json</code> Dynamically mapped at runtime via <code>productChatbotData</code>	Iterates through every fruit, vegetable, herb, and grain in the catalog: <pre> keywords: [product.name, product.slug] answer: "You can find {product.name} in the Produce Guide." link: `/produce-guide?search=\${encodeURIComponent(product.name)}` linkText: "View {product.name}" </pre>
Tier 2B: Farmers	<code>src/data/markets.json</code> Dynamically mapped at runtime via	Iterates through every neighborhood farmers market venue: <pre> keywords: [market.name, market.slug] </pre>

Market Index	marketChatbotData	<pre> answer: "View details and opening hours for {market.name}." link: `/markets/\${market.id}/\${market.slug }` linkText: "View {market.name}" </pre>
---------------------	--------------------------	---

The unified index is assembled via array spread concatenation: `const searchableChatbotData = [...chatbotData, ...productChatbotData, ...marketChatbotData];` providing exhaustive coverage across both general topics and specific products or venues.

8.3. Keyword Normalization & Longest-Match Heuristic Scoring Algorithm

To accurately discern user intent from unstructured natural language sentences, the assistant executes a multi-stage scoring algorithm:

1. **String Sanitization:** User queries are trimmed and converted to lowercase via `normalizeText(text) = text.toLowerCase()`, ensuring case-insensitive phrase matching.
2. **Candidate Scanning & Inclusion Check:** The engine iterates through every item in `searchableChatbotData`. For each trigger keyword registered under the item, it evaluates `questionToSearch.includes(normalizedKeyword)`.
3. **Longest-Keyword Scoring Heuristic (`score = keyword.length`):** If a keyword is contained within the query, its character length is evaluated:

```

4.
5. if (questionToSearch.includes(normalizedKeyword)
    && normalizedKeyword.length > longestKeyword) {
6.     longestKeyword = keyword.length;
7. }

```

Algorithmic Justification: This heuristic effectively prevents **intent collision**. For instance, if a user asks "Where is Downtown Farmers Market?", both the general rule "market" (length 6) and the

specific entity **"Downtown Farmers Market"** (length 23) match. Because $\$23 > \6 , the specific market entity achieves a significantly higher score, ensuring the assistant presents the exact market card instead of a generic directory link.

8. **Descending Priority Sorting:** Candidates are sorted in descending order of score: `matches.sort((a, b) => b.score - a.score)`.
9. **Duplicate Link Pruning:** To maintain visual cleanliness, duplicate target links are filtered out: `topAnswers.some((answer) => answer.link === item.link)`.
10. **Top-K Result Limiting:** The output is restricted to the top 4 candidates (`topAnswers.length === 4`), avoiding UI clutter on mobile displays.
11. **Graceful Fallback State:** If no keyword satisfies inclusion (`matches.length === 0`), `answers` remains empty and `hasAsked = true`, prompting the UI to display: *"I did not understand your question."*

8.4. Real-Time System Clock & Dynamic Schedule Synchronization

A distinctive capability of FreshFind is the tight coupling between the conversational assistant, the navigation clock, and real-time market schedules:

- **Persistent Header Clock:** Formatted via `getFormattedCurrentHourMinute()` in `src/utils/dateTimeUtils.js` using two-digit zero padding (`padStart(2, '0')`), the clock provides immediate temporal confirmation across every page.
- **Schedule Verification in Conversational Flows:** When a user queries opening hours or asks *"is market open now?"*, the chatbot routes them to the Market Directory or specific Market Details page. Here, the real-time engine compares client system time against the active timetable:

```
•  
• const currentMinutes = nowObj.getHours() * 60 +  
  nowObj.getMinutes();  
• const startMinutes =  
  timeToMinutes(todaySchedule.hours.start);  
• const endMinutes =  
  timeToMinutes(todaySchedule.hours.end);
```

- `return currentMinutes >= startMinutes && currentMinutes <= endMinutes;`

- **Proactive "Next Open Day" Computation:** If a market is closed during the time displayed on the clock, the system calculates the earliest upcoming operating session via `getNextOpenDay()`, greeting the visitor with: `"Closed today • Next open: [Day] at [Start Hour]"`.

8.5. Deep-Linking Navigation & Component Lifecycle Management

The chatbot component (`Chatbox.jsx`) provides a polished, accessible user experience:

- **Floating Launcher:** Positioned at `position: fixed; right: 20px; bottom: 20px; z-index: 1000`. Toggles between `"Chat"` and `"x"` with accessible `aria-label` tags.
- **Initial Welcome Experience:** Renders greeting text: *"Hello! What would you like to know about FreshFind?"* accompanied by quick-action suggestion chips (`"Find a market"`, `"Seasonal produce"`, `"Bookmarks"`).
- **Accessible Message Feed:** The message container incorporates `aria-live="polite"`, ensuring that screen readers automatically announce assistant replies upon submission.
- **Deep Action Buttons with Auto-Dismiss:** Each matched response displays a descriptive message alongside a direct React Router navigation link (`<Link to={item.link} onClick={closeChat}>`). When clicked, the application transitions instantly to the target URL (e.g., `/markets/1/downtown-farmers-market` or `/produce-guide?search=Tomato`) while `closeChat()` clears the query state and collapses the chat modal.